

When can the next work item reuse the buffer?

At the next output, the local loop counter returns to zero. The first shared buffer still belongs to an earlier work item until its asynchronous readers finish. Can the producer overwrite it now? Our existing example uses two buffers and three work items, partitioned as $[2,1]$, to put the first reuse exactly at this output boundary: q0 uses B0, q1 uses B1, and q2 returns to B0.

The example combines the persistent pipeline state in manual A4’s DeepGEMM mainloop with the resource-specific completion question in B2’s FA3 analysis. Its [current implementation and controls](#) run through Wasp with the required thread participation. It is a constructed protocol grounded in these mechanisms, not either complete upstream kernel.

The [existing interactive demonstration](#) remains the starting point: compare legal reuse with early release, then follow old reads through completion and release to the next write. Wrong input selection tests work correspondence independently; resetting pipeline state tests generation handoff. The deployed viewer contains the earlier run and earlier HB timings; those measurements retain their original baseline until new data is exported.

Now allow an iteration to skip an asynchronous update. Which parts of that read-to-reuse argument survive when accesses and waits disappear? The corpus rescale below exposes this next question because its supplied input fixes the branch to execute. It extends the existing investigation to a family of control choices; reliable coverage of that family is still the research obligation.

1. The question carried forward from the meetings

The team’s WEFT extension starts with a sampled execution that proposes barrier generations, then checks whether another admitted schedule can invalidate them. The transaction-counter analysis handles a phase that might finish earlier under another ordering. This is the established algorithmic setting for this work. Its internal phase checks remain necessary when the surrounding program contains loops and branches.

For our question, hold the kernel and shape fixed. Suppose that, on each loop iteration, tensor values decide whether a warp performs an asynchronous update. An execute choice creates accesses and a completion wait; a skip choice omits them. Both may reach the same later arrivals. Checking the always-execute sequence covers that sequence’s schedules. The other sequences require a coverage argument of their own.

Let $\mathbf{g}$ record the choices of the participating warps over the admitted loop occurrences, and let $\mathbf{G}$ be the declared set of such choice sequences. The target statement is simple:

For every $\mathbf{g}$ in $\mathbf{G}$, every execution admitted by that structure satisfies the specified synchronization and resource-safety conditions.

Choices within a collective operation must agree as its instruction requires. Choices in different warps or iterations may have further relations through the values being computed. Those relations need evidence. If $\mathbf{G}$ deliberately overapproximates them, safety for all of $\mathbf{G}$ also covers the feasible subset; a counterexample in $\mathbf{G}$ still needs a feasibility check before it is called a kernel bug. For schedule-dependent polling, guard outcomes constrain the admissible schedules. Treating choices independently gives an overapproximation whose counterexamples may be infeasible.

2. A small program with a real source

The corpus program [flash_attention_fwd_dense_bf16_h64_sm_100a.ptx](#) contains a repeated conditional TMEM update. Its source is [flash_attn/cute/flash_fwd_sm100.py](#). The following projection keeps the branch and common continuation. It groups four fragment updates and leaves the surrounding producer protocol as an explicit obligation.

```
for each work item:
    synchronize before reading the shared scale
    scale = load_shared_scale()
    execute = warp_ballot(scale < 1) != 0
    if execute:
        for each of four accumulator fragments:
            values = tcgen05.load(fragment)
            values = values * scale
            tcgen05.store(fragment, values)
        tcgen05.wait::st.sync.aligned
    mbarrier.arrive(A)
    mbarrier.arrive(B)
```

The collective instructions require valid warp participation. The correction warp issues operations; the asynchronous engine performs their memory effects. Receiving register values, completing stores, and permitting a different participant to reuse storage are distinct events. The skip arm issues none of these local TMEM operations and performs no store wait.

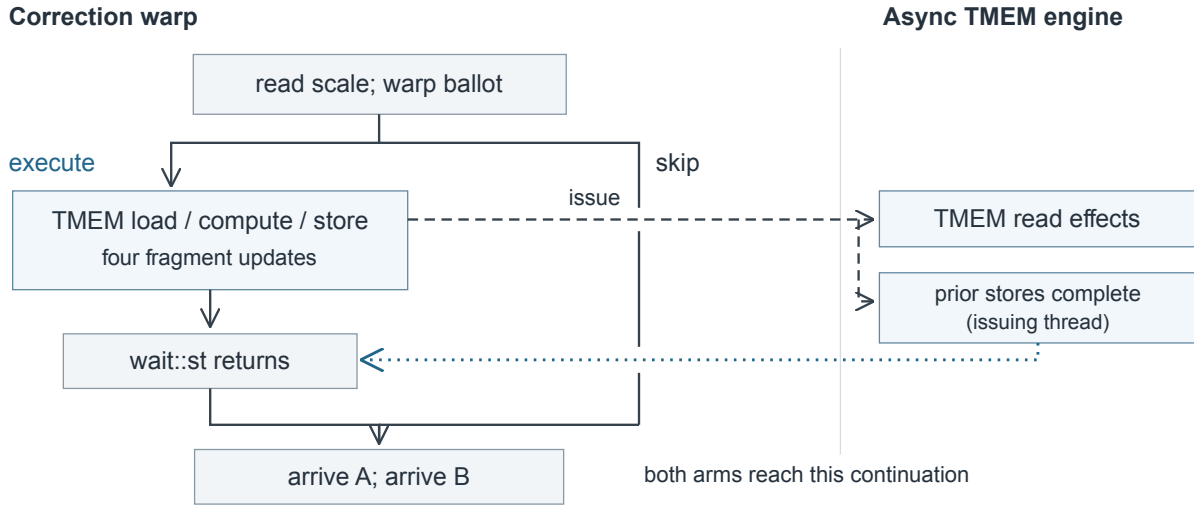


Figure 1: The correction warp takes one arm; the async engine performs only the issued operations. Solid arrows show control flow, dashed arrows issue effects, and the blue dotted arrow is the store-completion requirement of the wait. No cross-warp release-to-reuse edge is asserted.

The [current input](#) fixes the PTX branch at line 3892 to false, which executes rescaling. The [corpus report](#) states that dependence. Removing the supplied choice makes the current frontend stop at that branch. This identifies the missing coverage; it does not establish a race in the upstream kernel. Fresh runs at the unchanged full shape also complete for both always-execute and always-skip, under the same reported assumptions and duplicate-store warnings. These extremes do not settle mixed choices; neither may stand in for the family argument.

3. What a common exit does not tell us

If a correction thread enters with old stores still pending, an executing arm waits for them together with its new stores. A skipped arm provides no new completion guarantee. A common arrival therefore cannot, by itself, certify the same resource handoff on both paths.

The current [CFG analysis](#) resolves this particular store question for the pinned compiled loop: every reachable entry has an empty pending-store state, including paths through the outer backedge. Removing the drain breaks that invariant. This is a useful piece of the argument. It concerns stores issued by the analyzed thread and assumes legal execution of the modeled instructions.

Even equal empty exit states do not establish safety inside the region. Consider an abstract reader that has already issued an asynchronous read of buffer *O*, and a correction warp that can write *O* and drain its own stores. With no ordering between that old read and the new write, the write can happen while the read is still outstanding. Both operations may be finished by the time we inspect the region's exit.

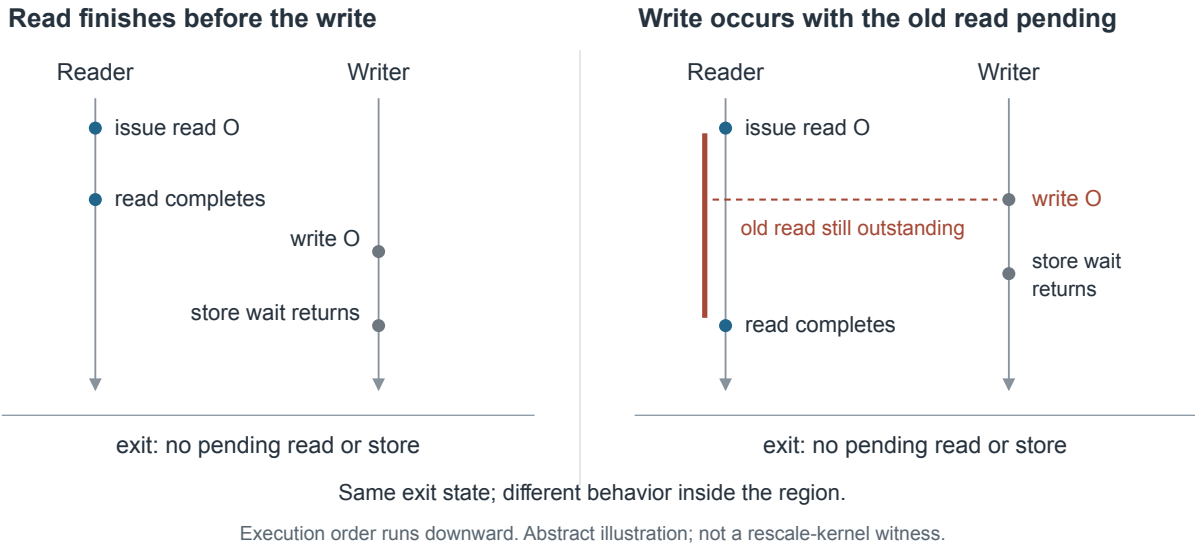


Figure 2: Two orders admitted by the constructed protocol with no read-to-write constraint. Both finish with no pending operations; the right order permits the new write while the old read remains outstanding. The diagram illustrates the loss caused by checking only the exit state, not an established rescale-kernel witness.

An exit summary must therefore retain violations encountered inside the region as well as unfinished work that can matter later. Nor may an unfinished read be forgotten merely because the next region does not touch its buffer: a later region can still overwrite it. Finally, `wait::st` does not itself establish all load-completion or collective-participation obligations. Those need the surrounding protocol argument developed below.

A native check now distinguishes two meanings of “finished.” In the extracted body, all 9216 memory/register effects precede the common arrival, yet 128 load read-completion nodes and 128 load-completion nodes have no path to that arrival. An explicit load wait closes them. This is a dependency-graph result, not a demonstrated bug in the original kernel.

An earlier [native template](#) uses exclusive single-warp TMEM, fixed addresses, an added load drain and a common blocking barrier. All eight three-round choices pass; missing-store-wait and pending-entry controls fail. It calibrates repetition when every effect and aggregate finishes before the next round,

but its full barrier cannot replace Attention’s actual multi-participant handoff.

The actual protocol gives a less restrictive route. There are four softmax warps and four correction warps. Each softmax warp publishes scale data to its paired correction warp through a named barrier; the publisher arrives without blocking, while correction waits. After the optional update, correction contributes to two shared barrier protocols. These are not a full-CTA blocking barrier, and each correction warp makes its own uniform execute/skip choice.

Keep the program with every update skipped as a common synchronization skeleton. Check that inserting updates preserves its instructions, operands, participants and primitive dependency paths. Empty store queues at entry and private scratch state make that preservation argument concrete for this body. Within the admitted Wasp event model, the preservation argument projects mixed executions onto the common skeleton and reuses its synchronization certificate. This result is relative to that certificate: the team’s latest specification still leaves lowering preservation and the cross-execution validity of its incremental synchronization checks as proof obligations. This extension does not discharge those underlying obligations. Optional memory accesses need an additional check.

For any potentially conflicting pair, ask a small question: **can we order this pair using only common events and the optional regions that create the pair?** If yes, that ordering exists whenever both accesses exist. If the path instead needs a third optional region’s wait, that third region can be skipped; the argument fails. This applies to lifecycle obligations as well as memory pairs. The implementation uses Wasp’s full conflict enumeration and reachability engine, grouping queries by the owners they require.

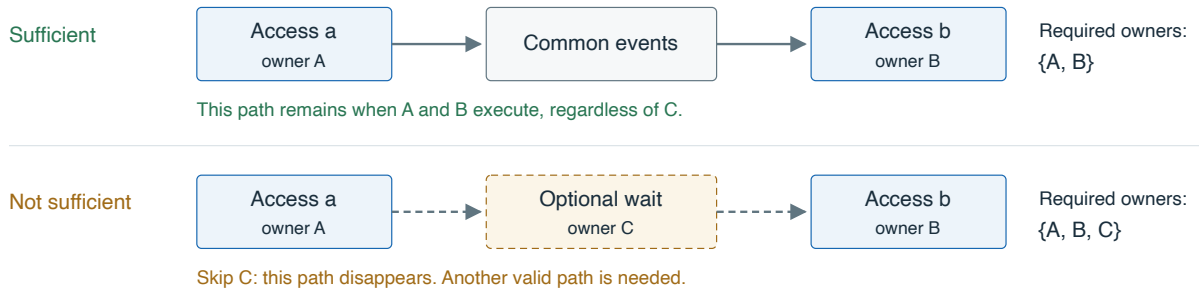
Write $O(e)$ for the singleton set containing the optional region that creates event e , or the empty set for a common event. An ordering path π is sufficient for a pair (a, b) if

$$\text{support}(\pi) \subseteq O(a) \cup O(b).$$

The support includes every optional event and conditional edge needed by the path. Whenever both endpoints exist, all of those requirements are present. This is a sufficient rule; a rejected path may have a different valid proof.

The pair exists whenever A and B execute.

Its proof must still work if every other optional region is skipped.



Schematic of the sufficient rule. A missing accepted path means unproved, not a demonstrated race.

Figure 3: A sufficient ordering path uses only common events and its endpoint owners. A path through another optional region can disappear when that region is skipped. This is an abstract rule diagram, not a recovered kernel witness.

This rule now passes on the original PTX with one CTA and two correction visits per warp: eight independent choices, or 256 choice vectors. It checks all retained conflict and lifecycle obligations together, rather than running 256 separate verifications. The inherited duplicate-TMA warning exemption remains explicit. The result is conditional on fixed common control and resolved memory footprints; it does not prove that changing tensor values preserves those premises. The missing-store-wait control produces an actual native lifecycle failure, with deallocation replayable before an outstanding store effect. An independent four-pair, single-generation operational model now exhausts all 16 warp choices, preserving nonblocking publication and independent async completions. Its schedules agree with the supported-path predictions; removing the store wait breaks completion ordering, while omitting publication or an A contributor breaks the common projection. This calibrates the abstract rule, not the full PTX or cross-generation refinement. The single-output end-to-end experiment below is complete; the output-boundary cohort reaches its budget before completing the ordering check.

4. The boundary state that earlier examples exposed

The two-buffer running example gives this state a familiar shape. There are two outputs with two and one work items. The local iteration restarts at the second output; the third work item reuses the first buffer. Producer and consumer progress independently. The producer must obtain permission to overwrite that buffer only after the consumer’s relevant asynchronous reads have completed.

work (output, local iteration):	(0,0)	(0,1)	(1,0)
continuous work ordinal:	0	1	2
buffer:	0	1	0
ready phase:	0	0	1

The [current native example](#) accepts continuous state and rejects the reset and early-release controls. Its source motivation is `deep_gemm/include/deep_gemm/impls/sm100_bf16_gemm.cuh`, specifically the pipeline state carried across scheduler outputs. This is a constructed protocol with that mechanism, not a complete GEMM.

The [real DeepGEMM corpus check](#) now tests this boundary directly. Across two outputs, all 32 MMA lanes advance their input stages from 0,1 to 2,3; resetting the cursor alone is rejected for a missing successful wait handoff. This uses a restricted two-CTA cluster with all 512 threads. The full cross-output launch exceeds 8 GiB, and four stage advances do not test the 12-stage ring’s first wrap.

Other existing examples constrain the same reasoning. Different QK and PV chains require waits that cover the particular resource being released. A tail can skip the next prefetch while retaining the current completion wait. Bulk copy groups distinguish source-read completion from destination-write completion. Three native mechanism slices now challenge the same projection and ordering rule: optional release changes synchronization and is rejected; an optional read wait cannot order a common buffer-reuse conflict; replacing full completion by read-only completion changes a common wait’s requirements and is rejected. These are checked applicability limits, not a general adapter for the original kernels. Work agreement remains separate: using the wrong input block can be race-free.

A fresh [real ThunderKittens B200 GEMM check](#) now makes the completion distinction concrete. An already compiled corpus entry writes four output slices through two shared buffers. Before slice 2 reuses the first buffer, `wait_group.read<1>` retires its earlier TMA reader. Changing only that wait to `<7>` permits the old read and reuse write in either order; both orders replay in Wasp’s native event model. The original completes race search without a diagnostic. The unchanged later wait still collects the old reader, but too late to protect this overwrite. This is a real PTX regression for an existing lowering capability, not new branch-family coverage or an upstream bug: the failing instruction is our negative

control. The run covers one task with all 512 threads and retains immediate visibility, representative election and failed cluster cancellation assumptions.

Variable-length attention separates an input gap from a new analysis problem. Supplying cumulative lengths, tensor maps and the output’s element strides lets a nonempty h128 corpus task complete default native checking. Missing output strides had forced a global memory-pair fallback and a timeout. Fixing that input is not an algorithmic speedup.

The same support rule now covers this task’s 16 independent warp choices. This body updates eight fragments and computes their addresses inside the branch, unlike the four-fragment h64 body above. Admission checks that those addresses are recomputed solely from immutable thread identity, fit the instruction’s integer range, and cannot alter common state. With those premises, the native family check orders all 2,354,144 potential conflicts and 5,253,120 required-order endpoint queries in five support groups, with no warning exemptions. Separate native runs of all 16 choices agree. This second profile covers one visit per correction warp, with common control and resolved resources fixed; it does not extend the theorem to arbitrary tensor inputs or loop lengths.

5. Where a sound abstraction could save work

The performance discussion has already moved beyond a naive all-pairs checker. Mengdi reported very large graphs, changed conflict selection to consider overlapping accesses before HB queries, and introduced C++, caching, pruning and batched queries. Those improvements reduce costs within an admitted graph. The meeting’s reported sizes and timings motivate the problem; they are not measurements produced by our current experiments.

Repeated choices create an additional opportunity: some ordering arguments may remain valid while other events change. For common query endpoints, a path built entirely from events and edges guaranteed to exist under every admitted choice proves the same order for every choice. An edge supplied only by an optional wait cannot be used that way. Combining all branch-specific edges would incorrectly make conditional ordering unconditional.

The endpoint-supported graph check is implemented and independently calibrated on 15,625 small graph families. Establishing its premises from a general program without certifying every sequence remains open. When a choice changes memory accesses, completion membership or generation matching, the affected obligations need a new check. This connects the abstraction to saved work; the admitted rescale family above now supplies a real-PTX instance with explicit common-state preservation and optional-access checks.

The single-output experiment compares complete choice enumeration with that family check. Both begin with the same PTX and input, and both pay frontend and verification costs. The family method also pays for projection and admission; these checks cannot be hidden outside its timer. The scale variable is the number of correction-branch visits per warp. A budget limit is reported as incomplete coverage, never extrapolated into a speedup. This measures whether the rule saves work after paying the cost of establishing its premises.

With one warmup and five measurements per completed arm, one visit (16 choices) takes a median 56.98 seconds by enumeration and 50.53 seconds by the family method: 1.13 times faster end to end. The measured ranges are 56.51–58.64 and 49.75–51.43 seconds. At two visits (256 choices), the family method completes in a median 103.73 seconds (103.28–105.65); enumeration reaches the 120-second limit after 25 choices. That result has no measured speedup ratio. At three visits, the family method also reaches 120 seconds. Larger sizes are not run, following the predeclared stop rule. Preparing and checking the premises is already substantial: three-visit setup consumes about 96 seconds before the

family ordering check. The rule reduces repeated reasoning, but this implementation has not solved end-to-end scalability. There is also a memory tradeoff: median sampled peak process-group RSS rises from 0.81 GiB to 4.07 GiB at one visit, and reaches 6.07 GiB at two visits for the family method. These peaks include its preparation processes; they are sampled every 50 ms plus process-listing latency, not exact allocator peaks.

A separate [guarded conflict-order optimization](#) preserves race existence under the same assumptions. Its counterexample shows why an exempted duplicate write cannot supply a transitive HB edge. The guard allows Wasp’s smaller check set only when that exemption cannot apply. A constructed N112 input improved from about 308 ms to 232 ms in native analysis. Three real-input comparisons gave no demonstrated benefit: two retained the old strategy, and one hit 8 GiB in both arms. These are separate results, not evidence of a real-corpus or multipath speedup.

Whether duplicate TMA stores and duplicate barrier initialization should be warnings or errors remains a team semantics question. Reporting assumptions also matters: an unsupported path, a synchronization failure and a model-based race witness communicate different facts. We must keep these questions visible because they change which conclusions and reductions are justified.

6. The next answer this work owes

The current answer covers independent warp/visit choices in the admitted fixed-control rescale model. Common synchronization is preserved, optional accesses remain in the complete check set, and ordering may use only owners that must be present. Native controls and independent small-state exploration challenge different parts of that argument.

Implementation continues in Wasp’s `research/loop-branch` branch. PTX and input pass through the existing frontend to `SourceProgram` and the native checker. The three existing [experiment entries](#), [reuse examples](#) and [efficiency records](#) retain the commands and detailed evidence. PhaseGuard supplies the earlier source analyses and counterexamples. It is not a runtime dependency.

The remaining research boundary is concrete: tensor-dependent feedback could change a common address or synchronization operand, invalidating the frozen model; other optional regions may mutate queues or synchronization directly. Those cases need their own checked premises. The two-output experiment passes progress and admission checks but times out during family ordering; a parent cleanup failure also leaves its final time and memory telemetry unavailable. No cross-output family safety result follows. A fresh source build and complete one-visit family replay pass without earlier generated artifacts. The current result does not establish arbitrary-loop safety or numerical equivalence.